

Police Shootings Analysis

Deep Dive

Working document for Salman Adnan

Generated from the actual source code and README in this repository

This document explains, in full, how this project works: what it loads, how it cleans the data, every chart it produces, and how the interactive dashboard is built on top of it. It is a working document for the project owner's own understanding and interview preparation, not part of the published website. Every technical term is defined the first time it appears. Nothing here is invented: every claim traces back to the actual code or the project's own README.

Contents

1	What this project is	3
2	Directory layout	4
3	The five data files	4
4	Two entry points, one shared cleaning problem	5
5	analyse_deaths.py: the static-chart script	5
5.1	Shape of the script	5
5.2	The chart-numbering mechanism	5
5.3	What the 24 charts cover	6
6	data.py: shared load/clean/filter logic	7
7	app.py: the interactive dashboard	9
7.1	Startup	9
7.2	Building the three charts	9
7.3	Serializing a Plotly figure to JSON	10
7.4	The two routes	10
8	templates/index.html: the dashboard page	10
9	Bugs actually hit while building this, and how they were fixed	11
9.1	plt.show() / fig.show() do not work headlessly	11
9.2	value_counts().reset_index() column naming changed across pandas versions . .	11
9.3	pycountry returns None for US state codes	12
9.4	Mixed date formats broke pd.to_datetime	12
9.5	scipy is a silent transitive dependency	12
9.6	A raw Plotly figure is not directly jsonify-able	12
10	Dependencies	12

11 Setup and usage	13
12 Licensing	13
13 Data and framing notes	13
14 Known limitations (from the README's own assessment)	14
15 Glossary	14

1 What this project is

This project analyzes a real, historical dataset published by *The Washington Post*: a list of fatal shootings by on-duty police officers in the United States. The copy of that dataset shipped in this repository (`Deaths_by_Police_US.csv`) covers **2,535 records** spanning 2015 through part of 2017. It is a fixed snapshot, not a live feed: running the project again does not pull new rows from the internet, it just re-reads the same CSV file that ships with the repo.

The project cross-references that fatality data against four datasets derived from the US Census: median household income, percentage of the population over 25 that completed high school, percentage of people below the poverty line, and racial makeup, all broken down by US city, mostly dated around 2015. A **dataset** here just means a table of rows and columns, usually stored as a **CSV file** (**CSV** stands for “comma-separated values,” a plain-text format where each line is a row and commas separate the columns).

The purpose (stated directly in the README) is a guided data-analysis exercise, originally built as a Udemy “100 Days of Code” project, to practice: joining several real-world datasets together, cleaning inconsistent real-world data, and producing the *same* chart using three different Python plotting libraries side by side, to compare how their APIs differ.

The repository ships **two** independent ways to look at the results:

1. `analyse_deaths.py`, a one-shot script: run it, it reads the 5 CSV files, cleans them, generates 24 charts, writes them to an `output/` folder, and exits. This is the original exercise code.
2. `app.py`, a small interactive web dashboard, built later on top of the same fatality CSV (not the four Census CSVs), using a web framework called **Flask** (defined in §7). It shows a clickable US map, lets you filter by state, year, and armed status, and updates three charts and a row of statistics live in the browser without reloading the page.

On the subject matter. This dataset records real deaths. The project (and this document) treats it the same way the README does: descriptive counts and correlations only, no causal claims, and an explicit warning that small subsets (for example, one race’s count in one city) can represent a handful of records. This document keeps that same neutral, factual framing throughout and does not add interpretation the README doesn’t already make.

2 Directory layout

```

police-shootings-analysis/
├── analyse_deaths.py    original static-chart script
├── app.py              Flask dashboard
├── data.py             shared load/clean/filter logic
├── templates/
│   └── index.html      dashboard page + JS
├── output/
│   └── chart_01.png ... chart_24.png    18 PNGs committed, 6 HTML files not
├── Deaths_by_Police_US.csv
├── Median_Household_Income_2015.csv
├── Pct_People_Below_Poverty_Level.csv
├── Pct_Over_25_Completed_High_School.csv
├── Share_of_Race_By_City.csv
├── requirements.txt
├── README.md
└── LICENSE

```

3 The five data files

All five CSVs are read with a specific text encoding called **windows-1252** (a character-encoding scheme, i.e. a mapping from bytes in a file to text characters, used by older Windows software; it is used here because the source files were not saved as **UTF-8**, the modern universal encoding, and reading them as UTF-8 would raise decoding errors or corrupt accented characters).

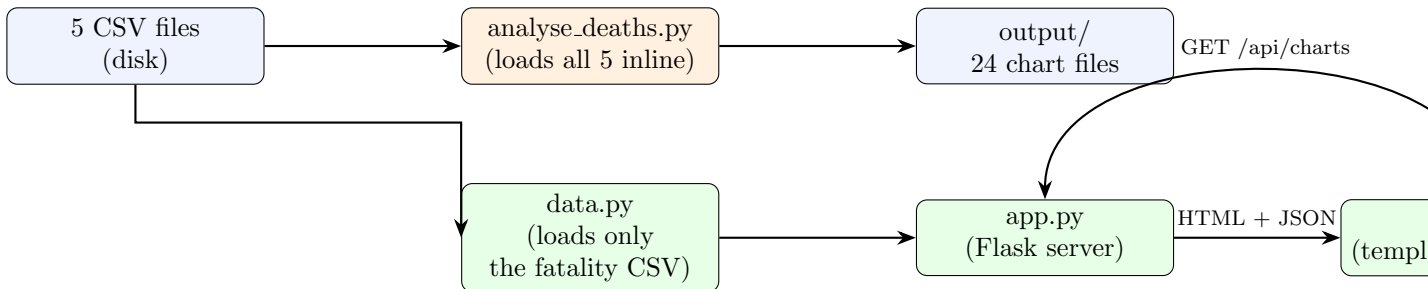
File	Rows (approx.)	What it holds
Deaths_by_Police_US.csv	2,535	One row per fatal shooting: id, name, date, manner of death, whether armed, age, gender, race, city, state, signs of mental illness, threat level, whether fleeing, whether the officer had a body camera.
Median_Household_Income_2015.csv	~20,000	Median household income per city.
Pct_People_Below_Poverty_Level.csv	~20,000	Percentage of each city's population below the poverty line.
Pct_Over_25_Completed_High_School.csv	~20,000	Percentage of each city's population aged 25+ who completed high school.
Share_of_Race_By_City.csv	~20,000	Percentage share of White, Black, Native American, Asian, and Hispanic population per city.

The four Census-derived files are joined against each other and, in places, against the fatality data by matching `Geographic Area` (state) and/or `City` text fields — there is no numeric database key linking them, just shared text columns, which is typical of public government data releases.

4 Two entry points, one shared cleaning problem

Both `analyse_deaths.py` and `app.py` need the fatality CSV in a usable shape before they can chart anything. The project solves this in two different ways depending on when the code was written:

- `analyse_deaths.py` is the original exercise script. It loads and cleans *all five* CSVs inline, top to bottom, with no functions of its own (beyond one small helper). It has not been touched since the dashboard was added.
- `data.py` is a new, small module written specifically so that `app.py` does not duplicate cleaning logic. It loads and cleans *only* the fatality CSV, using the exact same steps `analyse_deaths.py` uses on that file, so the dashboard’s numbers can never quietly drift from what the static script would compute on the same input.



5 analyse_deaths.py: the static-chart script

5.1 Shape of the script

This file is what is called a **procedural script**: a sequence of statements run top to bottom, in order, with no classes and almost no functions (a **class** bundles related data and behavior together; a **function** is a named, reusable block of code). It is roughly 390 lines. Aside from one helper function (`convert_country_code`) and a small counter function (`next_chart_path`), every chart is built inline, one after another.

Before anything else, the script forces **matplotlib** (a Python plotting library) into a **headless** backend:

```
import matplotlib
matplotlib.use('Agg')
```

“Headless” means running without a graphical display attached (as on a server, or inside an automated script with no window manager). **Agg** is a rendering backend that draws directly to an image file instead of trying to open a window. This one line is what makes the whole script runnable outside of an interactive notebook (see §9 for what broke before this fix existed).

5.2 The chart-numbering mechanism

```
_chart_index = 0
def next_chart_path(extension):
    global _chart_index
    _chart_index += 1
```

```
return os.path.join(output_dir, f'chart_{_chart_index:02d}.{extension}')
```

Every time a chart is saved, the script calls this function to get the next filename: `chart_01.png`, `chart_02.png`, and so on, up to `chart_24`. The counter is a **global variable** (a variable declared outside any function, shared and mutable across the whole file), which is why the function needs the `global` keyword to modify it. Matplotlib and seaborn charts are saved with `plt.savefig(...)` as `.png` image files; **Plotly** charts are saved with `fig.write_html(...)` as standalone `.html` files, because Plotly figures are interactive (zoomable, hoverable) and that interactivity only survives in an HTML+JavaScript file, not a flat image.

The 6 Plotly-generated HTML files are **not committed** to the repository. Each one embeds a full copy of the Plotly JavaScript library and inflates to roughly 5 MB, about 30 MB total for 6 files. Running the script regenerates them locally in under a minute, so the README treats them as reproducible build output rather than something worth storing in git. Only the 18 PNG files are committed, and this repository ships those 18 as an actual, already-run example (see Figure 1).

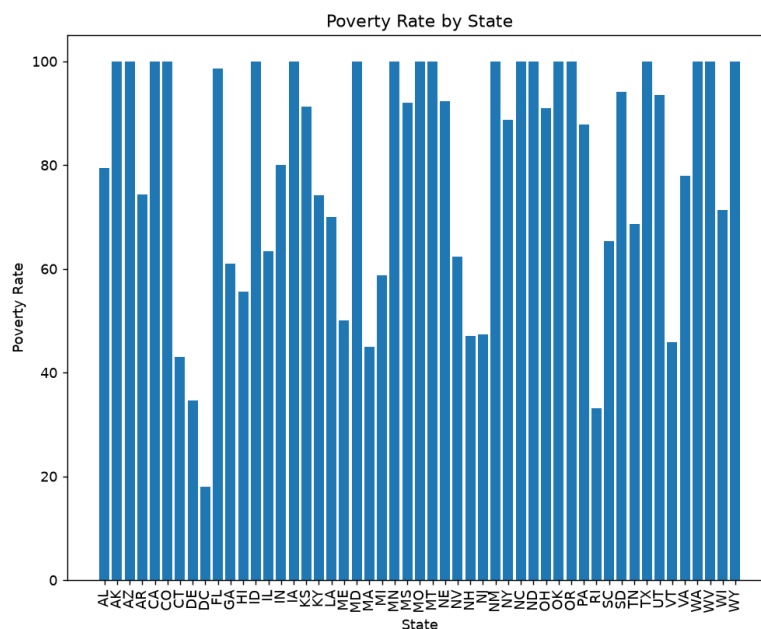


Figure 1: `chart_01.png` as actually produced by a run of `analyse_deaths.py`: poverty rate by US state, built with matplotlib.

5.3 What the 24 charts cover

Charts	Topic	Library / notes
1–3	Poverty rate by state	matplotlib, seaborn, plotly (same chart, three libraries)
4–6	High-school graduation rate by state	matplotlib, seaborn, plotly

7–9	Poverty rate vs. graduation rate relationship	seaborn line chart, plotly dual line chart, seaborn KDE joint plot (KDE , Kernel Density Estimate, is a smoothed curve estimating how a value's probability is distributed, here estimated by the <code>scipy</code> library under the hood)
10–11	Poverty vs. graduation regression	seaborn <code>lplot</code> and <code>regplot</code> , both draw a linear regression line (a straight best-fit line through scattered points) through the same data
12	Racial makeup by state	stacked/overlaid matplotlib bar chart (White, Black, Native American, Asian, Hispanic shares)
13	Gender split of people killed	pie/donut chart
14	Gender split, counted	bar chart
15	Age vs. manner of death	interactive plotly box plot (a box plot shows the median, quartiles, and outliers of a numeric distribution)
16	Whether the person was armed	bar chart, top 5 categories
17–18	Age distribution of people killed	histogram + KDE curve overlay, then a bar chart of counts by race
19	Signs of mental illness	pie chart
20	Top 10 cities by killings	bar chart (Figure 2)
21	Race breakdown within the top 10 cities	seaborn grouped bar chart
22–23	US choropleth map of killings by state	two plotly choropleths: one on a USA-only map projection, one on a world map projection (see §9.3 for why the second one is mostly illustrative)
24	Killings per year	bar chart (Figure 3)

A **choropleth map** is a map where regions (here, US states) are shaded according to a numeric value, in this case the number of recorded killings in that state.

6 data.py: shared load/clean/filter logic

This module exists purely so the dashboard does not silently diverge from the static script's cleaning rules. It exposes three functions:

```
def load_fatalities():
    df = pd.read_csv(os.path.join(BASE_PATH, 'Deaths_by_Police_US.csv'),
                     encoding='windows-1252')
    df = df.fillna(0)
    df['age'] = pd.to_numeric(df['age'], errors='coerce')
    df['date'] = pd.to_datetime(df['date'], format='mixed', errors='coerce')
    df['year'] = df['date'].dt.year
    return df
```

Line by line, for a reader new to **pandas** (a Python library for working with tabular data, organized

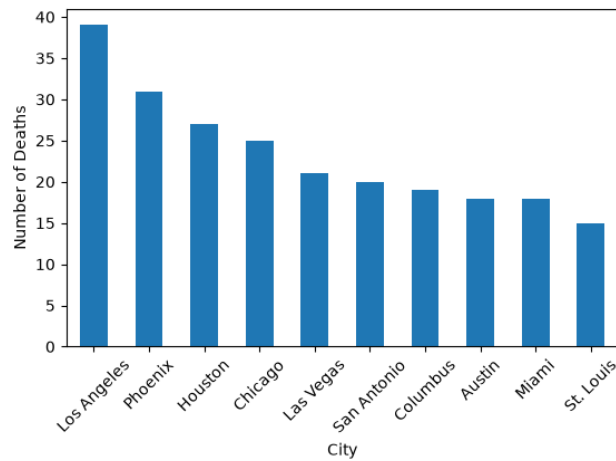


Figure 2: chart_20.png: top 10 cities by recorded killings.

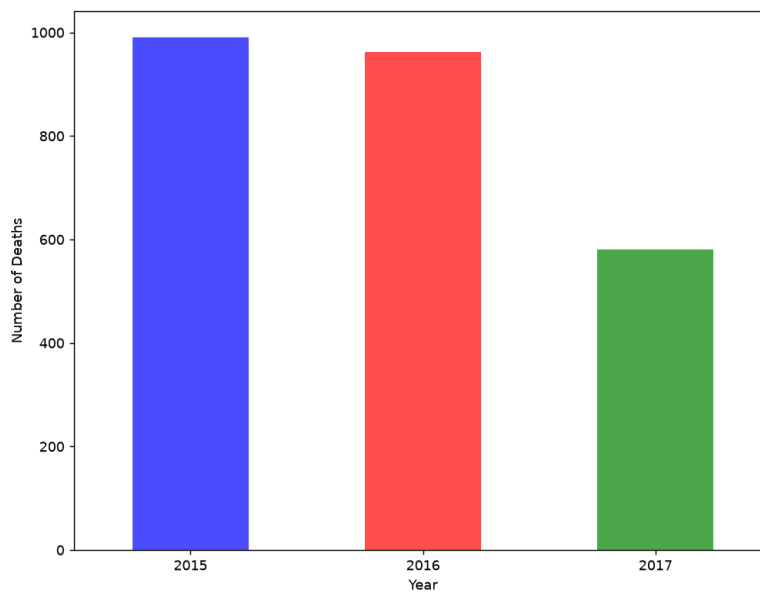


Figure 3: chart_24.png: killings per year (991 in 2015, 962 in 2016, 582 in a partial 2017, per the README’s verified run).

into a **DataFrame**, its core table-like data structure with named columns and a numeric or labeled row index):

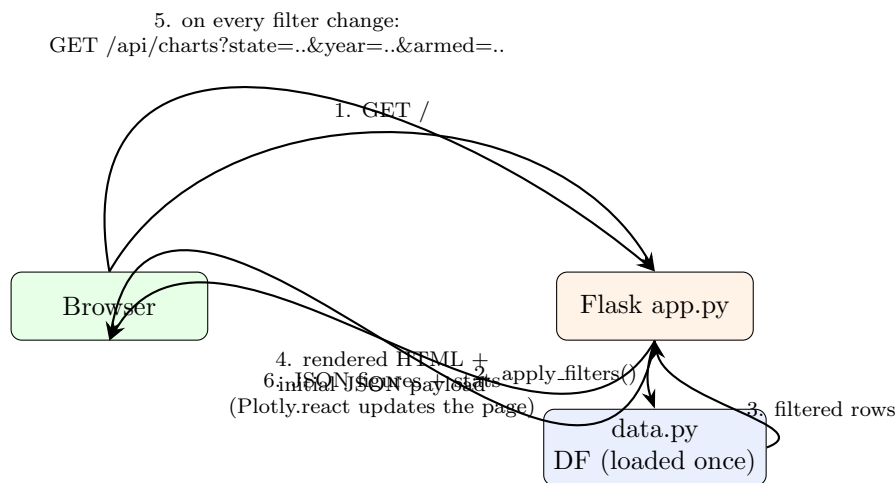
- `pd.read_csv(...)` reads the CSV file into a **DataFrame**.
- `df.fillna(0)` replaces every missing value (**NaN**, “Not a Number,” pandas’ placeholder for a missing cell) with the literal number/string 0.
- `pd.to_numeric(df[‘age’], errors=‘coerce’)` tries to convert the `age` column to actual numbers; `errors=‘coerce’` means any value that cannot be parsed as a number becomes **NaN** instead of crashing the program.
- `pd.to_datetime(..., format=‘mixed’, errors=‘coerce’)` parses the `date` column into real date objects, and `format=‘mixed’` tells pandas to infer each row’s date format individually rather than assume the whole column is one consistent format (see §9.4 for why that matters here specifically).
- `df[‘date’].dt.year` pulls just the year number out of each parsed date, stored in a new

`year` column.

`filter_options(df)` builds the three dropdown lists (states, years, armed types) directly from whatever values actually exist in the cleaned data, rather than a hand-maintained list, so a dropdown can never offer an option with zero matching rows. `apply_filters(df, state, year, armed)` takes the currently selected filter values and returns the subset of rows matching all three (any filter left as `None`, an empty string, or the literal string `'all'` is treated as “no restriction on this column”).

7 app.py: the interactive dashboard

Flask is a lightweight Python **web framework**: a library that handles the mechanics of receiving HTTP requests from a browser and sending back responses (HTML pages or JSON data), so the application code only has to describe what happens at each **route** (a URL path the server listens on) rather than implementing networking from scratch. `app.py` defines exactly two routes:



7.1 Startup

```
DF = load_fatalities()
STATES, YEARS, ARMED_TYPES = filter_options(DF)
```

The fatality data is loaded and cleaned exactly *once*, when the process starts, and kept in memory as `DF` for the lifetime of the server. Every incoming request reads from this same in-memory table (it is never mutated, only filtered into new, temporary DataFrames), so the dashboard never re-reads the CSV file from disk on a per-request basis.

7.2 Building the three charts

`build_choropleth`, `build_year_chart`, and `build_city_chart` each take the already-filtered DataFrame and construct one `plotly.graph_objects` figure:

- **Choropleth**: counts rows per state with `value_counts()`, then draws a `go.Choropleth` using `locationmode='USA-states'` (tells Plotly the location codes are US postal abbreviations, not country codes). The currently selected state (if any) is drawn with a thicker orange border (`marker_line_color/width` set per-state via a list comprehension) so the map visually highlights the active filter.

- **Year chart:** counts rows per year, sorted by year, as a plain bar chart.
- **City chart:** counts rows per city, keeps the top 10, and draws a horizontal bar chart.

`build_stats` computes four numbers from the same filtered rows: total record count, percentage with signs of mental illness, the most-affected state (with its count), and the most common armed status (with its count, excluding rows where `armed` was filled to 0 by `fillna`). Because it operates on the exact same filtered DataFrame the three charts were built from, the numbers shown in the stat row can never disagree with what the charts are visually displaying.

7.3 Serializing a Plotly figure to JSON

```
def fig_to_json(fig):
    return json.loads(fig.to_json())
```

A naive approach would hand `fig.to_dict()` straight to Flask's `jsonify()`. That fails at runtime, because Plotly figures carry numeric values computed by pandas/**numpy** (a numerical library pandas is built on) using numpy's own number types (`int64`, `float64`), and Flask's default JSON encoder does not know how to convert those to plain JSON numbers. Routing every figure through Plotly's own `fig.to_json()` first (which already knows how to serialize numpy types) and then parsing that string back into a plain Python dictionary with `json.loads()` avoids the problem entirely: by the time `jsonify()` sees the data, it is already plain Python `dict/list/int/float` values.

7.4 The two routes

```
@app.route('/')
def index():
    payload = build_payload(state=None, year=None, armed=None)
    return render_template('index.html', states=STATES, years=YEARS,
                           armed_types=ARMED_TYPES,
                           initial_payload_json=json.dumps(payload),
                           total_records=len(DF))

@app.route('/api/charts')
def api_charts():
    state = request.args.get('state') or 'all'
    year = request.args.get('year') or 'all'
    armed = request.args.get('armed') or 'all'
    return jsonify(build_payload(state=state, year=year, armed=armed))
```

`/` renders the full HTML page once, with an unfiltered payload embedded directly in the page (so the first paint needs no extra network round trip). `/api/charts` is a pure **JSON API endpoint** (a URL that returns machine-readable data rather than a full HTML page) that the page's JavaScript calls every time a dropdown changes or the map is clicked.

8 templates/index.html: the dashboard page

This is a single **Jinja2** template (Jinja2 is Flask's default templating engine: it lets a plain HTML file embed placeholders like `{{ total_records }}` and loops like `{% for s in states %}` that Flask fills in with real values before sending the page to the browser). Structurally:

- Inline CSS defines a light/dark color scheme using the `prefers-color-scheme` media query, so the dashboard matches the visitor's OS-level theme automatically.
- Three `<select>` dropdowns (state, year, armed status) are populated server-side from `states`, `years`, `armed_types`, the exact same lists `filter_options()` computed in `app.py`.

- A small inline `<script>` block owns all client-side behavior. It keeps no separate JavaScript file; everything is embedded in this one HTML file.

The script does three things:

1. `renderPayload(payload)` calls `Plotly.react(...)` for each of the three chart containers. `Plotly.react` is Plotly.js's method for *updating* an existing chart with new data/layout, as opposed to `Plotly.newPlot`, which would tear down and rebuild the chart's DOM node from scratch; using `react` is what keeps hover state, zoom level, and click listeners intact across a filter change.
2. `fetchAndRender()` reads the three dropdown values, builds a query string with **URLSearchParams** (a browser API for building `key=value&key2=value2` style URL query strings), calls `fetch('/api/charts?...')` (the browser's built-in HTTP request API), and passes the returned JSON to `renderPayload`.
3. A `plotly_click` event listener on the choropleth: when a state is clicked on the map, it sets the state dropdown to that state (or back to `'all'` if the same state is clicked twice) and calls `fetchAndRender()`, so clicking the map behaves exactly like picking that state from the dropdown.

The very first render, on page load, uses `initialPayload`, a JSON blob the Flask route already computed server-side and embedded directly into the page's `<script>` tag via `{{ initial_payload_json | safe }}`. This avoids one extra network round trip just to show the unfiltered view.

9 Bugs actually hit while building this, and how they were fixed

These are drawn directly from the README's own "Challenges" section; each one is a real failure encountered while writing or upgrading this code, not a hypothetical.

9.1 `plt.show()` / `fig.show()` do not work headlessly

The original script (written for an interactive Jupyter notebook, where a **Jupyter kernel** keeps a live Python process that can render figures into the notebook UI) called `plt.show()` after every matplotlib/seaborn chart and `fig.show()` after every plotly chart. Run as a plain script with no display attached, matplotlib's default backend has no window to draw into, and plotly's `fig.show()` raised `ValueError: Mime type rendering requires ipython`. Fixed by forcing the Agg backend (§7 above already covered what a backend is) and replacing every `.show()` call with `plt.savefig()` / `fig.write_html()` into the numbered `output/` folder.

9.2 `value_counts().reset_index()` column naming changed across pandas versions

Code like `df['state'].value_counts().reset_index(name='count')` used to produce a column literally named `index` on older pandas versions. On the pandas version installed here (3.0.3), the resulting column instead keeps the original series' name (`state`), so code that looked up `state_kills['index']` raised a `KeyError` (an error raised when a dictionary- or column-style lookup key does not exist). The same assumption broke the racial-makeup-by-state merge, where `merged_series.drop(columns='index')` failed outright because the column was never called `index` to begin with; it inherited the groupby key's name, `Geographic area`. Both spots were fixed to reference the real column names, and the racial-makeup merge was rewritten to rename each grouped Series explicitly before concatenating them, rather than trying to rename a generic column after the fact.

9.3 pycountry returns None for US state codes

The choropleth section runs every state abbreviation (WA, OR, etc.) through `convert_country_code`, a function meant to convert **ISO-3166 alpha-2** country codes (the two-letter country codes used worldwide, e.g. US, FR) into three-letter alpha-3 codes, using the `pycountry` library. US state postal codes are not ISO country codes, so `pycountry` returns `None` for every one of them, and the original code crashed with `AttributeError: 'NoneType' object has no attribute 'alpha_3'`. Fixed by adding a `None` check that falls back to the original code unchanged. Notably, the *first* choropleth in the script (the one using `locationmode='USA-states'`) does not need this conversion at all, since Plotly expects raw USPS state codes there; the conversion only affects the second map, which plots on a world-scope projection and is, per the README, “mostly illustrative” rather than a meaningful visualization of US-only data on a world map.

9.4 Mixed date formats broke `pd.to_datetime`

Most rows in the `date` column use M/D/YYYY (e.g. 2/1/2015), but some rows use DD/MM/YY (e.g. 13/01/15) instead. A plain `pd.to_datetime(df['date'])` call assumes one consistent format for the whole column and raised `ValueError: time data "13/01/15" doesn't match format`. Fixed with `format='mixed', errors='coerce': format='mixed'` tells pandas to infer each row's date format independently, and `errors='coerce'` turns any row it still cannot parse into `NaT` (“Not a Time,” pandas' missing- date placeholder) instead of crashing the whole load.

9.5 scipy is a silent transitive dependency

`df['age'].plot(kind='kde')` and `sns.jointplot(..., kind='kde')` both fail at runtime with `ModuleNotFoundError: No module named 'scipy'` unless `scipy` is installed separately: pandas' and seaborn's KDE plotting delegate to `scipy.stats.gaussian_kde` lazily (meaning the import only happens the moment a KDE chart is actually drawn, not at the top of the file), so the missing dependency does not surface until deep into a run. Fixed by adding `scipy` explicitly to `requirements.txt`.

9.6 A raw Plotly figure is not directly jsonify-able

Covered in full in §7.3 above: this is the one challenge specific to the dashboard rather than the original script, hit while wiring `app.py`'s `/api/charts` endpoint.

10 Dependencies

Package	Version	Role in this project
pandas	3.0.3	Loading CSVs into DataFrames; all cleaning, filtering, and aggregation (<code>value_counts</code> , <code>groupby</code> , <code>merge</code>).
matplotlib	3.11.0	One of the three plotting libraries compared; renders most static bar/pie/histogram charts.
seaborn	0.13.2	Built on top of matplotlib; used for the statistically-oriented charts (KDE joint plots, regression plots, grouped bar charts).
plotly	6.8.0	Interactive charting library; used for the standalone HTML charts in <code>analyse_deaths.py</code> and for every chart in the <code>app.py</code> dashboard (via its <code>graph_objects</code> and <code>express</code> submodules).

pycountry	26.2.16	Looks up ISO country-code metadata; used (with a fallback for misses) to try to convert state codes for the second, world-scope choropleth.
scipy	1.18.0	Provides the KDE (kernel density estimate) math that pandas/seaborn call into internally; never imported directly by name in this project's own code.
Flask	3.1.3	The web framework behind <code>app.py</code> : routing, request handling, Jinja2 templating, JSON responses.

11 Setup and usage

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

A **virtual environment** (`venv`) is an isolated copy of the Python interpreter and its installed packages, kept separate from the system-wide Python installation, so this project's exact dependency versions do not conflict with any other project's. No environment variables or external services are required: both entry points only read the CSV files already present in the project folder.

To run the dashboard: `python app.py`, then open `http://127.0.0.1:5000/` in a browser (127.0.0.1 is the loopback address meaning “this machine”; 5000 is the port Flask listens on by default). To run the original static script: `python analyse_deaths.py`, which writes 24 files into `output/` (created automatically if missing) and exits.

12 Licensing

The project's LICENSE file uses the **PolyForm Strict License 1.0.0**. In plain terms, as actually written in that license text: it grants a license to use the software for any permitted purpose, explicitly *excluding* redistributing the software or creating modified/derivative works from it. Noncommercial use, personal research/study/hobby use, and use by charitable, educational, public-research, public-safety/health, environmental, or government institutions are all listed as permitted purposes regardless of funding source. It does not grant any right to sublicense or transfer the license to someone else. This replaced an earlier MIT license on this repository; this document reflects the license as it currently stands, PolyForm Strict, not MIT.

13 Data and framing notes

As stated directly in the README: all charts here report descriptive counts and correlations already present in the joined data. Nothing in this project establishes **causation** (a cause-and-effect relationship) as opposed to mere **correlation** (two things trending together without either necessarily causing the other). Some cross-tabulations (for example, race broken down within the top 10 cities) are computed from a small enough number of matching rows that individual bars can represent only a handful of records, meaning small subsets should not be read as statistically strong.

Verified, not estimated, from an actual run of the script: 2,535 total fatality records; 2,428 male and 107 female; 633 of 2,535 rows (about 25%) recorded signs of mental illness; the five most common **armed** values are gun (1,398), knife (373), vehicle (177), unarmed (171), and undetermined (117);

the five states with the most recorded killings are California (424), Texas (225), Florida (154), Arizona (118), and Ohio (79); year counts are 991 (2015), 962 (2016), and 582 (a partial year, 2017).

14 Known limitations (from the README’s own assessment)

- `analyse_deaths.py` is still one ~390-line procedural script; none of its chart-building code is broken out into functions, unlike `data.py/app.py`, which show what that split looks like for the fatality-only path.
- The dashboard’s filters only apply to the fatality data, not the four Census CSVs, because joining city names across those datasets is not unique per state and was judged not solid enough to build filtering logic on top of.
- `convert_country_code` is described in the README as a leftover from a template meant for country-level choropleths; it is unnecessary for the first, USA-scoped map and only partially meaningful for the second, world-scoped one.
- There is no schema or dtype (data-type) validation at load time: a malformed CSV would fail deep inside a chart-building block with a confusing traceback rather than a clear error up front.
- The dataset is a fixed historical snapshot (2015 through part of 2017) with no mechanism to pull an updated copy of the Washington Post database; the README is explicit that this should not be read as current data.

15 Glossary

Term	Meaning
CSV	Comma-separated values; a plain-text table format, one row per line.
DataFrame	Pandas’ core table-like data structure: rows and named columns.
NaN / NaT	Pandas’ placeholders for a missing number / missing date-time value.
Headless	Running without a graphical display attached.
Backend (plotting)	The underlying rendering engine a plotting library draws with (e.g. Agg draws to a file instead of a window).
Choropleth	A map with regions shaded by a numeric value.
KDE	Kernel Density Estimate; a smoothed curve estimating a value’s probability distribution.
Web framework	A library handling HTTP request/response mechanics so application code only defines behavior per route.
Route	A URL path a web server listens on.
JSON	JavaScript Object Notation; a plain-text data format for structured data, used here for API responses.
Jinja2 template	An HTML file with embedded placeholders/loops a web framework fills in server-side.
Virtual environment	An isolated Python interpreter + package set for one project.
Correlation vs. causation	Two things trending together, versus one actually causing the other; this project only ever measures the former.